LegacyNexus AI

Azure Deployment Runbook

Step-by-step roadmap, commands and configuration for deploying LegacyNexus AI to an Azure free trial subscription, with Docker images built locally (Rancher Desktop) and pushed to Azure Container Registry.

Item	Value
Prepared	5 November 2026
Environment	dev (Azure free trial, `-Sizing free`)
Live URL	https://lnx-dev-web.jollyground-fcf6c803.eastus.azurecontainerapps.io
Resource group	`LegacyNexus` (East US); PostgreSQL in Central US
Repository files	`infra/azure/main.bicep`, `infra/azure/deploy.ps1`, `backend/Dockerfile`, `frontend/Dockerfile`, `frontend/nginx.conf`, `DEPLOYMENT.md`

Contents

1. Roadmap at a glance
2. Architecture
3. Phase 1 - Prerequisites and tools
4. Phase 2 - Verify the application locally
5. Phase 3 - Sign in to Azure and check the subscription
6. Phase 4 - One-time Azure preparation
7. Phase 5 - Pre-flight checks
8. Phase 6 - Deploy (create resources, build, push, roll out)
9. Phase 7 - Verify the deployment
10. Phase 8 - Post-deployment configuration (AI keys, budget)
11. Phase 9 - Updating the application
12. Reference A - Azure resources created
13. Reference B - Configuration and secrets
14. Reference C - deploy.ps1 parameters
15. Reference D - Issues found during this deployment and fixes
16. Reference E - Troubleshooting and logs
17. Reference F - Clean-up and cost control
18. Reference G - Command cheat sheet

1. Roadmap at a glance

Follow the phases in order. Phases 1, 3 and 4 are needed only once per machine or subscription; for later releases only Phase 9 is needed.

Phase	What you do	Time	Frequency
1. Prerequisites	Azure account, Azure CLI, Rancher Desktop (dockerd), Git, PowerShell	30 min	Once
2. Local check	`docker compose up` and confirm the app and health checks work	10 min	Each release
3. Sign in	`az login`, confirm free trial subscription and Owner role	2 min	Each session
4. Azure prep	Install Bicep, register resource providers	3 min	Once
5. Pre-flight	Check PostgreSQL region, existing resources, dry-run validation	5 min	First deploy
6. Deploy	Run `deploy.ps1` with `-LocalBuild`	25-30 min first, 5-10 min later	Each release
7. Verify	`/api/health/ready`, get admin password, sign in	5 min	Each release
8. Configure	AI model + embedding keys in Settings, budget alert	10 min	Once
9. Update	Rerun the same deploy command after code changes	5-10 min	Each release

2. Architecture

```
Browser
   |  HTTPS
   v
web  (Container App, public)        nginx + React UI; forwards /api/* to the API
   |  internal HTTPS (HTTP/1.1)
   v
api  (Container App, internal)      FastAPI backend
   |
   +--> PostgreSQL Flexible Server 16 + pgvector   (Central US)
   |       +- database legacynexus          platform data, code graph, Code Q&A + vectors
   |       +- database legacynexus_docqa    Document Q&A + vectors
   +--> Azure Files share  -> /app/data     uploaded code, caches
   +--> Key Vault                           secrets, read via managed identity
   +--> Service Bus queue "analysis-jobs"
             |
             v
        worker (Container Apps Job)        analysis / semantic indexing, scales 0..N
```

The same two images run locally (`docker compose`) and on Azure. All Azure connection settings are injected as environment variables by `infra/azure/main.bicep`; secrets come from Key Vault through the managed identity `lnx-dev-id`. **No connection strings, keys or passwords are placed in the images, Dockerfiles or .env files.**

Background jobs: the API puts a small message (`{"kind": "analyze", "project_id": 5}`) on the Service Bus queue. KEDA starts the `worker` job when messages are waiting; the worker runs `python -m app.worker`, completes the message and exits. Failed jobs are retried up to 5 times and then dead-lettered. Locally, `JOB_BACKEND=inline` runs the same jobs inside the API process instead.

3. Phase 1 - Prerequisites and tools

3.1 Azure account

Requirement	Details
Subscription	Azure free trial (`FreeTrial_2014-09-01`) or Pay-As-You-Go. The trial gives US$200 credit for 30 days plus 12 months of selected free services (including PostgreSQL B1ms).
Role	**Owner** on the subscription (or Contributor + User Access Administrator). The template creates role assignments for the managed identity.
After 30 days	Upgrade the trial to Pay-As-You-Go (portal > Subscriptions > Upgrade), otherwise Azure disables all resources. Remaining credit and free services are kept.

3.2 Tools on the deployment machine (Windows)

Tool	Purpose	Check command
Azure CLI 2.60+	Sign in, create resources, read Key Vault	`az version`
Bicep (via Azure CLI)	Compile the infrastructure template	`az bicep version`
Rancher Desktop	Build and push Docker images (provides `docker`)	`docker info`
Git	Pull the latest code before deploying	`git --version`
PowerShell 5.1+	Run `deploy.ps1`	`$PSVersionTable.PSVersion`

3.3 Rancher Desktop settings

- Install Rancher Desktop, start it and wait until the status bar shows it is ready (first start: several minutes).
- **Preferences > Container Engine:** `dockerd (moby)`. `containerd` does not provide the `docker` command that `deploy.ps1` and `az acr login` need.
- **Preferences > Kubernetes:** untick Enable Kubernetes (not needed; saves memory).
- Restart VS Code / open a new terminal so the `docker` command is on PATH. Rancher installs it under `%LOCALAPPDATA%\Programs\Rancher Desktop\resources\resources\win32\bin`.
- Keep Rancher running during the whole deployment.

```
docker info --format '{{.ServerVersion}} {{.OSType}}/{{.Architecture}}'
# expected: 29.x.x linux/x86_64
```

A warning such as "No swap limit support" from `docker info` is harmless. `deploy.ps1` now ignores warnings and only stops on real failures (see Reference D).

3.4 Project files used

File	Role
`backend/Dockerfile`	API + worker image: Python 3.12-slim, installs `requirements.txt` and `requirements-azure.txt` (Service Bus, identity), pre-downloads tree-sitter grammars, runs as non-root user, port 8000, health check on `/api/health`.
`frontend/Dockerfile`	Web image: Node 20 builds the React app (`VITE_API_BASE_URL=/api`), nginx 1.27 serves it on port 80, health check on `/healthz`.
`frontend/nginx.conf`	Forwards `/api/*` to `${BACKEND_URL}` using HTTP/1.1; unlimited upload size, 1-hour proxy timeouts, security headers.
`infra/azure/main.bicep`	Every Azure resource, role assignment and app setting.
`infra/azure/deploy.ps1`	Creates resources, generates secrets, builds and pushes images, rolls out.
`.dockerignore` (backend, frontend)	Keep `.env`, `node_modules`, local data and tests out of the images.

4. Phase 2 - Verify the application locally

Before every cloud release, confirm the code runs in containers locally. `docker-compose.yml` starts PostgreSQL 16 + pgvector, the backend and the web UI. Run from the project root (`C:\legacynexus`).

```
git pull                                    # latest code
copy backend\.env.local.example backend\.env   # first time only, then edit SECRET_KEY etc.
docker compose up -d --build                # build and start db, backend, frontend
docker compose ps                           # all three should be Up (healthy)
```

Check the health endpoints (or open them in a browser):

```
Invoke-WebRequest http://localhost:8080/healthz          -UseBasicParsing
Invoke-WebRequest http://localhost:8080/api/health       -UseBasicParsing
Invoke-WebRequest http://localhost:8080/api/health/ready -UseBasicParsing
```

`/api/health/ready` must report `ok: true` for `database`, `vectors`, `docqa_database`, `docqa_vectors`, `storage` and `jobs`. Then open http://localhost:8080 and sign in with `BOOTSTRAP_ADMIN_EMAIL` / `BOOTSTRAP_ADMIN_PASSWORD` from `backend/.env` (local demo: `admin@legacynexus.ai`).

```
docker compose logs -f backend    # follow backend logs
docker compose down               # stop (data kept in volumes)
docker compose down -v            # stop and delete local data
```

> The local demo password is refused by the app outside development, so it can never reach Azure. After 5 wrong sign-in attempts an account is locked for 15 minutes (restarting the backend clears it).

5. Phase 3 - Sign in to Azure and check the subscription

```
az login                                                  # browser sign-in
az account list -o table                                  # all subscriptions you can see
az account set --subscription "Azure subscription 1"      # if you have more than one
az account show --query "{name:name, user:user.name}" -o table
```

Confirm it is the free trial (or Pay-As-You-Go) and not a company production subscription:

```
$sub = az account show --query id -o tsv
az rest --method get `
  --url "https://management.azure.com/subscriptions/$sub`?api-version=2022-12-01" `
  --query "{quota:subscriptionPolicies.quotaId, limit:subscriptionPolicies.spendingLimit}" -o table
# expected: FreeTrial_2014-09-01   On
```

Confirm you are Owner:

```
$me = az ad signed-in-user show --query id -o tsv
az role assignment list --assignee $me --all --query "[].{role:roleDefinitionName}" -o table
```

> If a browser cannot open on the machine, use `az login --use-device-code`, open https://login.microsoft.com/device and enter the code shown. Finish within 15 minutes or the code expires.

6. Phase 4 - One-time Azure preparation

Install Bicep and register the resource providers the template uses. Registration takes 1-2 minutes.

```
az bicep install
$providers = 'Microsoft.App','Microsoft.ContainerRegistry','Microsoft.DBforPostgreSQL',
             'Microsoft.KeyVault','Microsoft.ServiceBus','Microsoft.Storage',
             'Microsoft.OperationalInsights','Microsoft.ManagedIdentity'
foreach ($p in $providers) { az provider register -n $p }
# add 'Microsoft.CognitiveServices' if you will use -WithAzureOpenAI

az provider list --query "[?namespace=='Microsoft.DBforPostgreSQL'].registrationState" -o tsv
# repeat until every provider shows Registered
```

7. Phase 5 - Pre-flight checks

7.1 Where PostgreSQL is allowed

Free trial subscriptions are often blocked from creating PostgreSQL in popular regions. In this deployment East US, East US 2, South Central US and West US 2 were **restricted**; Central US, West US 3, Canada Central and North Central US were **allowed**.

```
foreach ($r in 'eastus','centralus','westus3','canadacentral','centralindia') {
  $reason = az postgres flexible-server list-skus -l $r --query "[0].reason" -o tsv
  "{0,-14} {1}" -f $r, $(if ($reason) { "RESTRICTED: $reason" } else { "allowed" })
}
```

Use an allowed region for `-DbLocation`. The other resources can stay in the resource group's region.

7.2 Existing resources in the resource group

```
az group list -o table
az resource list -g LegacyNexus --query "[].[name, type]" -o tsv
```

This deployment found `LegacyNexusRegistry` (Container Registry, Basic, empty), `lnx-dev-logs` and `lnx-dev-id` already present. The registry is reused with `-ExistingAcrName`; `lnx-dev-logs` and `lnx-dev-id` have the template's own names, so the template adopts them.

7.3 Dry-run validation (creates nothing)

```
$me = az ad signed-in-user show --query id -o tsv
az deployment group validate -g LegacyNexus -f infra\azure\main.bicep -p `
  namePrefix=lnx env=dev bootstrapAdminEmail=<admin-email> allowSignup=false `
  deployerPrincipalId=$me deployerPrincipalType=User sizing=free deployOpenAI=false `
  existingAcrName=LegacyNexusRegistry dbLocation=centralus `
  dbAdminPassword=Validate0nly!x123456 appSecretKey=validate-only-secret-key-0123456789abcdef `
  bootstrapAdminPassword=Validate0nly!x1 --query properties.provisioningState -o tsv
# expected: Succeeded
```

The passwords above are placeholders used only for validation; nothing is stored.

8. Phase 6 - Deploy

Run from the project root in PowerShell, with Rancher Desktop running and `docker` on PATH. This is the exact command used for the current environment:

```
cd C:\legacynexus
Set-ExecutionPolicy -Scope Process Bypass

./infra/azure/deploy.ps1 -ResourceGroup LegacyNexus -Location eastus -DbLocation centralus `
  -Env dev -AdminEmail abhi8712shek@gmail.com -Sizing free `
  -LocalBuild -GenerateAdminPassword -ExistingAcrName LegacyNexusRegistry
```

For a brand-new subscription with no existing resources, a typical first command is:

```
./infra/azure/deploy.ps1 -ResourceGroup rg-legacynexus-dev -Location eastus -DbLocation centralus `
  -Env dev -AdminEmail <admin-email> -Sizing free -LocalBuild -GenerateAdminPassword
```

8.1 What the script does

Step	Output line	What happens	Time
1	`==> Resource group ...`	Creates the resource group (or confirms it exists).	seconds
2	`==> First deployment: generating secrets`	First run only. Generates the PostgreSQL password, app SECRET_KEY and (with `-GenerateAdminPassword`) the admin password. They are written only to Key Vault.	seconds
3	`==> Creating infrastructure`	First run only. Deploys `main.bicep` with placeholder images: identity, logs, Key Vault + secrets, PostgreSQL + 2 databases + pgvector, Service Bus + queue, storage + share, Container Apps environment, api, web, worker, role assignments.	10-15 min
4	`==> Building images locally and pushing`	`az acr login`, `docker build --platform linux/amd64` for backend and frontend, `docker push` both, tagged with a timestamp (e.g. `202611050821`).	5-10 min
5	`==> Rolling out images`	Redeploys `main.bicep` with the new image tags; Container Apps start a new revision.	2-4 min
6	`LegacyNexus AI is deployed: https://...`	Prints the web URL, the readiness URL and (first run with `-GenerateAdminPassword`) the Key Vault command for the admin password.	-

> Later runs skip steps 2 and 3: secrets are read back from Key Vault and never rotated, so data and stored API keys stay valid.

9. Phase 7 - Verify the deployment

```
$web = "https://lnx-dev-web.jollyground-fcf6c803.eastus.azurecontainerapps.io"
Invoke-WebRequest "$web/healthz"          -UseBasicParsing   # 200 -> web app is up
Invoke-WebRequest "$web/api/health"       -UseBasicParsing   # 200, profile=cloud
Invoke-WebRequest "$web/api/health/ready" -UseBasicParsing   # every component ok=true
```

Result recorded for this deployment:

Component	Kind	Where
database	PostgreSQL	`lnx-dev-pg-r2lzxahfb2phi.postgres.database.azure.com/legacynexus`
vectors	pgvector	`legacynexus` - table `lnx_chunk_vectors`
docqa_database	PostgreSQL	`.../legacynexus_docqa`
docqa_vectors	pgvector	`legacynexus_docqa` - table `lnx_doc_vectors`
storage	Mounted file share	`/app/data/storage`
jobs	Azure Service Bus	`lnx-dev-sb-r2lzxahfb2phi.servicebus.windows.net`

Get the admin password and sign in:

```
az keyvault secret show --vault-name lnxdevkvr2lzxahfb2phi `
  -n bootstrap-admin-password --query value -o tsv
```

- Open the web URL and sign in with the `-AdminEmail` address and that password.
- The first request after idle can take 10-30 seconds: on `-Sizing free` the apps scale to zero.
- You can change the password after the first sign-in (the API supports `/api/auth/change-password`).

10. Phase 8 - Post-deployment configuration

10.1 Where API keys and connections are configured

What	Where it is configured	Action needed
Database, vectors, Service Bus, file share, Key Vault	Automatically by `main.bicep` (env vars on the container apps)	None
App secret, DB password, admin password	Key Vault, generated by `deploy.ps1`	None
AI chat model + key	In the app: **Settings > AI models**	Enter provider and key
Embedding model + key	In the app: **Settings > Semantic search**	Enter provider and key
Email for password resets (optional)	`SMTP_*` entries in `backendEnv` in `main.bicep`	Add and redeploy

10.2 AI model (chat) - Settings > AI models

Supported providers: Azure OpenAI, OpenAI, Anthropic, Gemini, Ollama, OpenAI-compatible. For Azure OpenAI:

Field	Example value
Provider	Azure OpenAI
Endpoint	`https://<resource-name>.openai.azure.com`
Deployment	`gpt-4o-mini`
API version	`2024-10-21`
API key	Key 1 from the Azure OpenAI resource (Keys and Endpoint)

10.3 Embeddings - Settings > Semantic search

Same provider and endpoint; deployment `text-embedding-3-small`. After saving, build the semantic index for a project (it runs as a background job through Service Bus).

10.4 Optional: let the script create Azure OpenAI

```
./infra/azure/deploy.ps1 ... -WithAzureOpenAI [-OpenAILocation eastus]
```

This creates the Azure OpenAI account with `gpt-4o-mini` and `text-embedding-3-small`, stores the key in Key Vault (`ai-api-key`) and sets `AI_*` / `EMBEDDING_*`. These are applied only while nothing is configured in Settings, so an administrator's choice is never overwritten. Free trials may have no Azure OpenAI quota.

10.5 Budget alert

Portal > **Cost Management > Budgets** > Add: scope the subscription, monthly amount (e.g. US$10), email alert at 80% and 100%.

11. Phase 9 - Updating the application

```
cd C:\legacynexus
git pull
docker compose up -d --build        # optional local check (Phase 2)
Set-ExecutionPolicy -Scope Process Bypass
./infra/azure/deploy.ps1 -ResourceGroup LegacyNexus -Location eastus -DbLocation centralus `
  -Env dev -AdminEmail abhi8712shek@gmail.com -Sizing free `
  -LocalBuild -ExistingAcrName LegacyNexusRegistry
```

- `-GenerateAdminPassword` is not needed on updates (it only applies to the first run).
- Each run builds new images with a new timestamp tag, pushes them and rolls out a new revision.
- Database schema migrations run automatically when the API starts (both databases).
- Re-check `/api/health/ready` after every update.

12. Reference A - Azure resources created

Resource group `LegacyNexus`. Unique suffix generated by the template: `r2lzxahfb2phi`.

Resource	Name	Region	Size (free sizing)	Purpose
Managed identity	`lnx-dev-id`	East US	-	Pulls images, reads secrets, uses the queue (no keys)
Log Analytics	`lnx-dev-logs`	East US	PerGB2018, 30 days	Container logs
Container Registry	`LegacyNexusRegistry` (`legacynexusregistry.azurecr.io`)	East US	Basic	Images `legacynexus-api`, `legacynexus-web`
Key Vault	`lnxdevkvr2lzxahfb2phi`	East US	Standard, RBAC	Secrets
PostgreSQL Flexible Server	`lnx-dev-pg-r2lzxahfb2phi`	Central US	Burstable B1ms, 32 GB, v16	Databases `legacynexus`, `legacynexus_docqa`, pgvector
Service Bus	`lnx-dev-sb-r2lzxahfb2phi` / queue `analysis-jobs`	East US	Basic	Background jobs; dead-letter after 5 tries
Storage account	`lnxdevstr2lzxahfb2phi` / share `legacynexus-data`	East US	Standard LRS	Mounted at `/app/data`
Container Apps environment	`lnx-dev-env`	East US	Consumption	Hosts the apps
Container App (public)	`lnx-dev-web`	East US	0.5 vCPU / 1 GiB, 0-3 replicas	Website
Container App (internal)	`lnx-dev-api`	East US	1 vCPU / 2 GiB, 0-5 replicas	API
Container Apps Job	`lnx-dev-worker`	East US	2 vCPU / 4 GiB, 0-10 runs, 6 h timeout	Worker

Role assignments

Who	Role	On
`lnx-dev-id`	AcrPull	Container Registry
`lnx-dev-id`	Key Vault Secrets User	Key Vault
`lnx-dev-id`	Azure Service Bus Data Sender / Data Receiver	Service Bus namespace
Person deploying	Key Vault Secrets Officer	Key Vault (lets later runs read secrets)

Approximate monthly cost on free sizing

Resource	Cost
PostgreSQL B1ms + 32 GB	Free for 12 months on a free account (750 h/month)
Container Apps	Within the monthly free grant at light use (scale to zero)
Container Registry Basic	About US$5/month
Service Bus Basic, Key Vault, Storage	A few cents/month
Log Analytics	Free up to 5 GB/month
AI provider	Pay per token, billed by the provider you configure

13. Reference B - Configuration and secrets

Set by `main.bicep` on `lnx-dev-api` and `lnx-dev-worker` (`backendEnv`). Read by `backend/app/config.py`.

Variable	Value on Azure	Source
`DEPLOYMENT_PROFILE`	`cloud`	Fixed
`ENVIRONMENT`	`production`	Fixed (refuses demo secrets)
`SECRET_KEY`	secret ref `secret-key`	Key Vault
`DB_HOST`	`lnx-dev-pg-r2lzxahfb2phi.postgres.database.azure.com`	PostgreSQL server
`DB_NAME` / `DB_USER`	`legacynexus` / `lnxadmin`	Template
`DB_PASSWORD`	secret ref `db-password`	Key Vault
`DB_SSLMODE`	`require`	Fixed
`DOCQA_DB_NAME`	`legacynexus_docqa`	Same server
`VECTOR_STORE` / `DOCQA_VECTOR_STORE`	`pgvector` / `pgvector`	Fixed
`CONVERSATION_CONTEXT`	`auto`	Fixed
`STORAGE_DIR`	`/app/data/storage`	Azure Files mount
`JOB_BACKEND`	`servicebus`	Fixed
`SERVICEBUS_NAMESPACE`	`lnx-dev-sb-r2lzxahfb2phi.servicebus.windows.net`	Service Bus
`SERVICEBUS_QUEUE`	`analysis-jobs`	Template
`AZURE_CLIENT_ID`	client id of `lnx-dev-id`	Managed identity
`CORS_ORIGINS` / `APP_BASE_URL`	`https://lnx-dev-web.<env-domain>`	Web app
`ALLOW_SIGNUP`	`false` (unless `-AllowSignup`)	Script
`BOOTSTRAP_ADMIN_EMAIL`	`-AdminEmail` value	Script
`BOOTSTRAP_ADMIN_PASSWORD`	secret ref `bootstrap-admin-password`	Key Vault
`AI_*`, `EMBEDDING_*`	Only with `-WithAzureOpenAI`	Azure OpenAI + Key Vault
`WORKER_RUN_ONCE`	`true` (worker only)	Template
`BACKEND_URL` (web app)	`https://lnx-dev-api.internal.<env-domain>`	Template

Key Vault secrets

Secret	Content	Created by
`secret-key`	Signs sign-in tokens and encrypts stored AI keys. Never rotate.	deploy.ps1 (first run)
`db-password`	PostgreSQL admin password	deploy.ps1 (first run)
`bootstrap-admin-password`	First administrator's password	deploy.ps1 (first run)
`ai-api-key`	Azure OpenAI key	main.bicep with `-WithAzureOpenAI`

Never put cloud values in `backend/.env`, in the Dockerfiles (`ENV ...`) or in source control. `.env` is excluded from the images by `.dockerignore` and from Git by `.gitignore`.

14. Reference C - deploy.ps1 parameters

Parameter	Required	Meaning
`-ResourceGroup`	Yes	Resource group to create or use
`-AdminEmail`	Yes	First administrator's sign-in email
`-Location`	No (`westeurope`)	Region of the resource group and most resources. Must match an existing group's region.
`-Env`	No (`prod`)	Label in resource names, e.g. `dev`
`-NamePrefix`	No (`lnx`)	Prefix for resource names
`-Sizing`	No (`standard`)	`free` = B1ms 32 GB, apps scale to zero, Service Bus Basic
`-LocalBuild`	No	Build with local Docker and push (needed on free trials: cloud builds are blocked)
`-GenerateAdminPassword`	No	First run: generate the admin password into Key Vault instead of prompting
`-ExistingAcrName`	No	Use an existing registry in the group instead of creating one
`-DbLocation`	No	Region for PostgreSQL only, when `-Location` is restricted
`-WithAzureOpenAI`	No	Create Azure OpenAI with chat + embedding deployments
`-OpenAILocation`	No	Region for Azure OpenAI
`-AllowSignup`	No	Allow self sign-up (default invite-only)

15. Reference D - Issues found during this deployment and fixes

#	Problem	Cause	Fix (now in the repository)
1	Cloud image build would fail	Free-trial subscriptions cannot use ACR Tasks (`az acr build`)	New `-LocalBuild` switch: `docker build --platform linux/amd64` + `docker push`
2	Script waited for a typed admin password	`Read-Host` prompt	New `-GenerateAdminPassword` switch; password stored only in Key Vault
3	Second container registry would be created	Template always created its own registry	New `existingAcrName` template parameter / `-ExistingAcrName`
4	PostgreSQL: "Provisioning is restricted in this region"	Free-trial region restriction in East US	New `dbLocation` parameter / `-DbLocation centralus`
5	Script stopped on "WARNING: No swap limit support"	Windows PowerShell 5.1 with `ErrorActionPreference=Stop` treats tool warnings on stderr as fatal	Script uses `Continue` and checks each command's exit code (`Assert-Ok`)
6	`/api/*` returned **426 Upgrade Required** on Azure	nginx proxies with HTTP/1.0 by default; Container Apps ingress rejects it	`proxy_http_version 1.1;` in `frontend/nginx.conf`
7	Azure CLI sign-in cache unreadable from another session	Windows encrypts the CLI token cache per user session	Sign in again with `az login` (or `--use-device-code`) in the session that runs the deploy

16. Reference E - Troubleshooting and logs

Symptom	Action
`location is restricted` (PostgreSQL)	Check regions (Phase 5.1); add `-DbLocation <allowed region>`
`TasksOperationsNotAllowed` / ACR build blocked	Use `-LocalBuild`
`-LocalBuild needs Docker Desktop installed`	Install Rancher Desktop; open a new terminal
`Docker is installed but not running`	Start Rancher Desktop and wait until ready
`Invalid resource group location`	`-Location` must equal the existing group's region
Quota error (vCPU / Container Apps)	Try another region or request a quota increase in the portal
Azure OpenAI creation fails	Rerun without `-WithAzureOpenAI`; set a provider in Settings
Script blocked by execution policy	`Set-ExecutionPolicy -Scope Process Bypass`
`/api/...` returns 426	Ensure `proxy_http_version 1.1;` is in `frontend/nginx.conf`, redeploy
`/api/health/ready` shows ok=false	Read the API logs below; check the named component
Analysis stays queued	Check worker executions and Service Bus queue / dead-letter count
Sign-in fails repeatedly	Locked 15 minutes after 5 failures; re-read the Key Vault password
First page very slow	Normal on free sizing (scale from zero); wait 10-30 s

Logs and status commands

```
az containerapp logs show -g LegacyNexus -n lnx-dev-api --follow
az containerapp logs show -g LegacyNexus -n lnx-dev-web --follow
az containerapp revision list -g LegacyNexus -n lnx-dev-api -o table
az containerapp job execution list -g LegacyNexus -n lnx-dev-worker -o table
az servicebus queue show -g LegacyNexus --namespace-name lnx-dev-sb-r2lzxahfb2phi `
  -n analysis-jobs --query "countDetails" -o json
az deployment group show -g LegacyNexus -n legacynexus-dev --query properties.outputs
az deployment operation group list -g LegacyNexus -n legacynexus-dev `
  --query "[].{res:properties.targetResource.resourceName, state:properties.provisioningState}" -o table
az acr repository show-tags -n LegacyNexusRegistry --repository legacynexus-api -o table
```

17. Reference F - Clean-up and cost control

- Set a budget alert (Phase 8, 10.5).
- Delete old image tags occasionally to stay within the Basic registry's 10 GB.
- To remove the whole environment (cannot be undone):

```
az group delete -n LegacyNexus --yes
az keyvault purge -n lnxdevkvr2lzxahfb2phi    # optional: Key Vault is soft-deleted for 30 days
```

> Deleting the resource group deletes the databases and uploaded files. Export anything you need first.

18. Reference G - Command cheat sheet

```
# --- every release ----------------------------------------------------------
cd C:\legacynexus
git pull
docker info --format '{{.ServerVersion}}'            # Rancher running?
az login                                              # if not signed in
Set-ExecutionPolicy -Scope Process Bypass
./infra/azure/deploy.ps1 -ResourceGroup LegacyNexus -Location eastus -DbLocation centralus `
  -Env dev -AdminEmail abhi8712shek@gmail.com -Sizing free `
  -LocalBuild -ExistingAcrName LegacyNexusRegistry
$web = "https://lnx-dev-web.jollyground-fcf6c803.eastus.azurecontainerapps.io"
Invoke-WebRequest "$web/api/health/ready" -UseBasicParsing
```

```
# --- admin password ------------------------------------------------------
az keyvault secret show --vault-name lnxdevkvr2lzxahfb2phi -n bootstrap-admin-password `
  --query value -o tsv

# --- local ---------------------------------------------------------------
docker compose up -d --build
docker compose logs -f backend
docker compose down
```